

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего
образования «Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

**Использование расширенного контекста и поэтапных
рассуждений для генерации SQL-запросов
с помощью больших языковых моделей**

Студент,
группы 5130201/30003

_____ Четвергов И.С

Руководитель,
доцент к.т.н.

_____ Попов С. Г.

«_____» _____ 2026 г.

Санкт-Петербург
2026

Содержание

1	Введение	3
2	Постановка задачи	3
3	Методология	4
3.1	Retrieval-Augmented Generation	4
3.2	Chain-of-Thought Prompting	4
3.3	Семантическая оценка: LLM-as-a-Judge	4
4	Реализация	4
4.1	Ограничения системы	5
4.2	Общая архитектура	5
4.3	Представление схемы базы данных	6
4.4	Двухиндексный семантический поиск	7
4.5	Граф схемы и структурное обогащение	7
4.6	Двухэтапная генерация SQL и роли языковой модели	8
	Список литературы	11

1 Введение

Реляционные системы управления базами данных (РСУБД) являются фундаментальной технологией для хранения и обработки структурированной информации. Традиционное взаимодействие с ними осуществляется посредством декларативного языка запросов SQL (Structured Query Language). Однако эффективное использование этого инструмента требует от специалиста не только владения синтаксисом, но и глубокого понимания специфической схемы данных, что создает высокий порог вхождения.

Для преодоления барьера взаимодействия человека с вычислительными системами активно развивается направление интерфейсов на естественном языке (Natural Language Interfaces, NLI). В рамках этой широкой области выделяется специализированный класс систем — естественно-языковые интерфейсы к базам данных (Natural Language Interfaces to Databases, NLIDB). Их ключевой задачей является интерпретация произвольного вопроса пользователя и его трансляция в формальный SQL-запрос (Text2SQL).

В последние годы широкое распространение при построении NLIDB получили большие языковые модели (Large Language Models, LLM). Несмотря на способности к обобщению и пониманию семантики языка, решения на основе «чистых» LLM сталкиваются с рядом ограничений: лимитом контекстного окна и невозможностью вместить полную информацию о схемах данных. Это часто приводит к фактическим ошибкам (hallucinations): модель генерирует синтаксически верный запрос, который тем не менее обращается к несуществующим таблицам или столбцам, использует неверные условия соединения или возвращает семантически некорректный результат — такие ошибки не обнаруживаются на уровне синтаксического разбора и проявляются только при выполнении запроса.

Для преодоления дефицита контекста применяется метод RAG (*Retrieval-Augmented Generation*), предполагающий предварительное извлечение релевантной информации перед этапом генерации. В задаче Text-to-SQL это включает выбор наиболее значимых фрагментов схемы базы данных — таблиц, столбцов и их описаний. Такой подход позволяет снизить вероятность обращений к несуществующим объектам схемы.

Однако одного расширения контекста недостаточно для корректной обработки сложных запросов, требующих многошагового логического вывода, агрегирования и построения соединений между таблицами. Для решения этой проблемы применяется стратегия Chain-of-Thought (CoT), направленная на явную декомпозицию рассуждений модели перед генерацией итогового SQL-запроса.

2 Постановка задачи

Пусть задана реляционная база данных $\mathcal{D} = \{T_1, \dots, T_n\}$, где каждая таблица T_i описывается множеством столбцов $C_i = \{c_{i1}, \dots, c_{ij}\}$ и множеством внешних ключей $F_i \subseteq C_i \times C_j$.

Пусть задан естественно-языковой запрос пользователя

$$q \in \mathcal{Q}.$$

Задача Text2SQL заключается в построении функции

$$f : \mathcal{Q} \times \mathcal{D} \rightarrow \mathcal{S},$$

где $\mathcal{S}$ — множество синтаксически корректных SQL-запросов.

3 Методология

3.1 Retrieval-Augmented Generation

RAG разделяет генерацию ответа на два этапа: поиск релевантных фрагментов во внешней базе знаний и генерация с учётом найденного контекста [1]. Документы предварительно кодируются в плотные векторы с помощью нейронной сети-энкодера и индексированы для быстрого поиска ближайших соседей [9]. При поступлении запроса он кодируется тем же энкодером, после чего по косинусному сходству извлекаются наиболее релевантные фрагменты и передаются модели вместе с вопросом [12].

Это позволяет не передавать в контекст полную схему базы данных, а ограничиться только релевантными таблицами и примерами [4].

3.2 Chain-of-Thought Prompting

Chain-of-Thought побуждает модель генерировать промежуточную цепочку рассуждений перед формированием итогового ответа [2]. Формально, вместо моделирования $P(S | Q)$ система моделирует $P(S | C, Q) \cdot P(C | Q)$, где C — последовательность шагов, объясняющая логику решения [3]. Это позволяет разбить работу на два независимых этапа: планирование JOIN-цепочки и непосредственную генерацию SQL, сужая пространство поиска на каждом шаге.

В данной работе CoT реализован как два последовательных обращения к модели с разделёнными ролями.

3.3 Семантическая оценка: LLM-as-a-Judge

В качестве основной метрики применяется LLM-as-a-Judge [8]: языковая модель оценивает синтаксическую корректность, точность привязки к схеме и соответствие намерению пользователя, возвращая структурированный вердикт. Такой подход позволяет оценивать семантическую близость запроса к эталону без необходимости точного синтаксического совпадения и без запуска на реальных данных.

Ключевое ограничение подхода — качество оценки ограничено возможностями модели-судьи. Если судья и генератор — одна и та же модель, они разделяют одни и те же слабые стороны. Реализация и анализ поведения судьи описаны

4 Реализация

Задача Text-to-SQL решается без дообучения модели — веса фиксированы, управление качеством генерации осуществляется исключительно через контекст и структуру рассуждений. Поскольку модель предобучена на SQL, она владеет синтаксисом как фиксированным априорным знанием. Обозначим через $\mathcal{C}$ пространство допустимых SQL-конструкций. Синтаксический априор:

$$P_{\text{LLM}}(s) \propto \mathbf{1}[s \in \mathcal{C}]$$

считается фиксированным — задача сводится к управлению контекстом и структурой рассуждений.

Схема базы данных представляется ориентированным графом $G = (V, E)$, где $V = \{T_i\} \cup \{c_{ij}\}$ — узлы (таблицы и столбцы), $E = E_{\text{table-column}} \cup E_{\text{foreign-key}}$ — рёбра принадлежности и внешних ключей.

В рамках RAG вводится процедура ретривала Retrieve, которая по запросу пользователя q извлекает релевантное подмножество схемы:

$$\mathcal{R}(q) = \text{Retrieve}(q, \mathcal{D}), \quad \mathcal{R}(q) \subseteq \mathcal{D}.$$

Модель получает не всю схему, а только найденный контекст. Итоговая функция генерации принимает вид:

$$\hat{s} = f(q, \mathcal{R}(q)).$$

Для декомпозиции сложных запросов вводится промежуточная переменная рассуждения $t \in \mathcal{T}$, где $\mathcal{T}$ — пространство возможных цепочек рассуждений (Chain-of-Thought). Генерация SQL факторизуется:

$$P(s \mid q, \mathcal{R}(q)) = \sum_{t \in \mathcal{T}} P(s \mid t, q, \mathcal{R}(q)) \cdot P(t \mid q, \mathcal{R}(q)).$$

Практически используется аппроксимация через наиболее вероятное рассуждение:

$$\hat{t} = \arg \max_{t \in \mathcal{T}} P(t \mid q, \mathcal{R}(q)), \quad \hat{s} = \arg \max_{s \in \mathcal{S}} P(s \mid \hat{t}, q, \mathcal{R}(q)).$$

Итоговое качество определяется тремя компонентами: предобученным знанием P_{LLM} , качеством ретривала $\mathcal{R}$ и корректностью цепочки рассуждений $\hat{t}$.

4.1 Ограничения системы

Текущая реализация предполагает ряд ограничений на схему базы данных и архитектуру пайплайна.

Граф G должен быть связным: между любыми двумя таблицами должен существовать путь через внешние ключи — изолированные таблицы без FK-связей не могут быть включены в JOIN-цепочку алгоритмом Дейкстры.

Схема предполагается ациклической: циклические зависимости приводят к неоднозначности при построении пути и не обрабатываются корректно.

Retrieval-тексты, граф и значения `hub_score` написаны вручную для каждой таблицы — система не масштабируется на произвольную схему без ручной подготовки метаданных.

4.2 Общая архитектура

Система работает в два режима. При старте выполняется офлайн-подготовка: схема индексируется в два векторных индекса и строится граф внешних ключей. При каждом запросе пользователя запускается онлайн-пайплайн из четырёх шагов: семантический ретривал, структурное обогащение через граф, планирование JOIN-цепочки и генерация SQL. Архитектура пайплайна показана на рисунке 1.

В качестве языковой модели используется квантованная версия Qwen2.5-Coder 14B, запускаемая локально через Ollama [11] с контекстным окном 8192 токена. Развёртывание выполнено на MacBook Air M3 с 16 ГБ оперативной памяти без облачных ресурсов. При подаче неструктурированной схемы модель этого размера стабильно галлюцинирует имена таблиц и строит неверные JOIN-цепочки — каждый элемент пайплайна является отдельным ответом на конкретный класс этих ошибок.

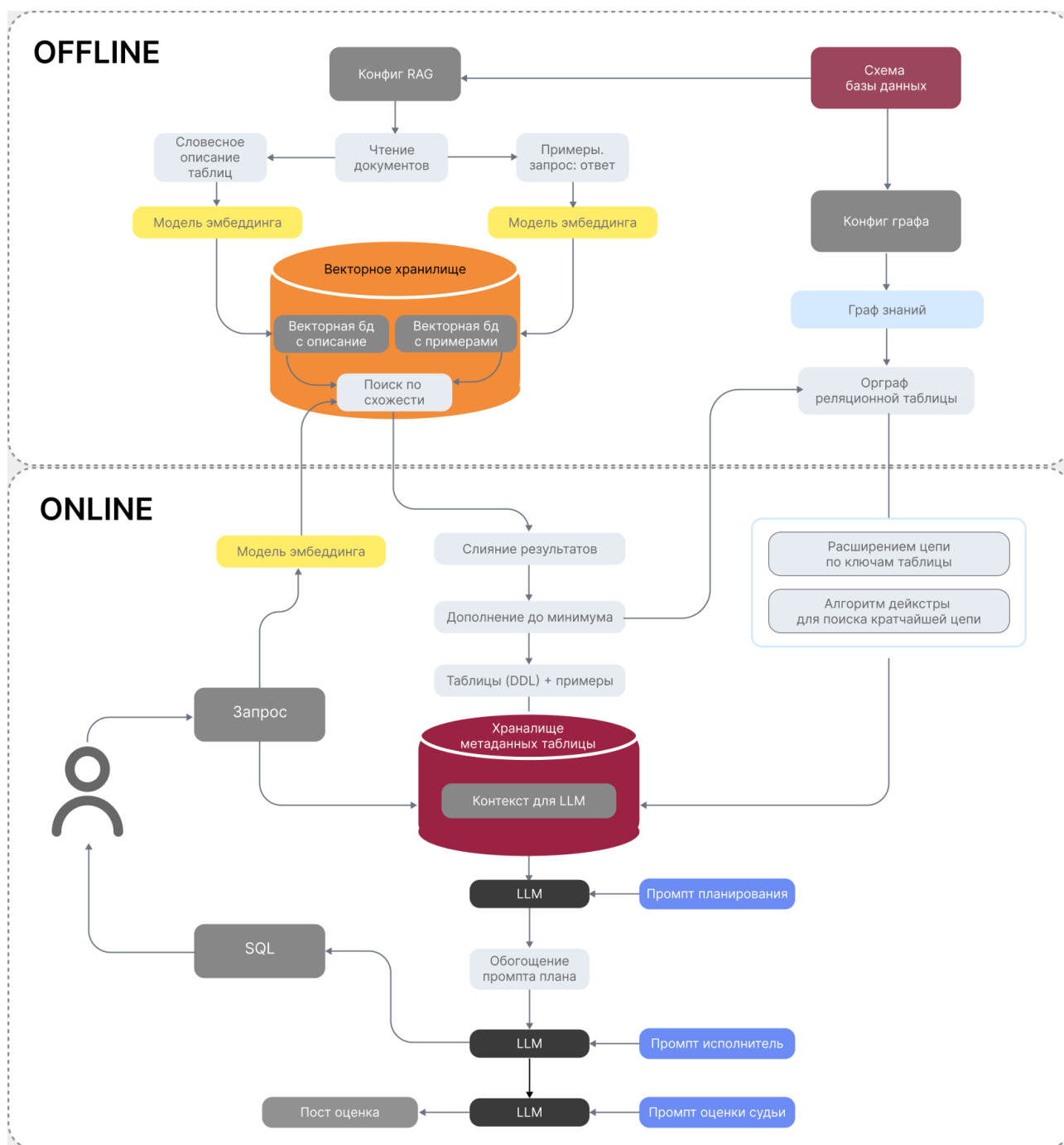


Рис. 1: Схема архитектуры системы

4.3 Представление схемы базы данных

Предметная область — платформа для проведения соревнований по машинному обучению: участники получают датасеты, отправляют предсказания и получают оценку по зафиксированной метрике. Схема содержит 12 таблиц: пользователи, соревнования, участия, отправки, оценки, таблица лидеров, датасеты, файлы, конфигурация соревнования, метрики, типы задач и ноутбуки участников.

Каждая таблица описывается тремя представлениями с разными функциями.

Retrieval-текст — описание на естественном языке, написанное как набор фраз из реальных пользовательских вопросов. Для *participation* это «участие пользователя в соревновании», «количество участников», «статус участия». Индексируется векторной моделью и участвует в семантическом поиске.

Context-mesct — компактный DDL с аннотациями столбцов, передаётся языковой модели после нахождения таблицы ретривалом. Позволяет уместить несколько таблиц в одно контекстное окно без потери семантики.

Граф внешних ключей $G = (V, E)$ строится из конфигурационного файла при старте сервиса. Рёбра хранят направление FK-связи, имена столбцов, кардинальность и предпочтительный тип JOIN. Каждый узел имеет атрибут `hub_score` — чем он ниже, тем охотнее таблица используется как связующее звено. Узлы с `hub_score` ≥ 5 считаются листовыми и исключаются из автоматического расширения контекста.

Все три представления написаны вручную — это основное ограничение масштабируемости системы.

4.4 Двухиндексный семантический поиск

Ретривал использует два отдельных индекса — таблиц и примеров запросов. Такое разделение позволяет управлять вкладом каждого источника независимо через веса: найденный пример подсказывает паттерн JOIN и агрегации, тогда как описание таблицы даёт структурный контекст.

Алгоритм 1. Семантический ретривал

Вход: вопрос q ; индексы $\text{idx}_{\text{tables}}$ и $\text{idx}_{\text{examples}}$; веса w_{tables} , w_{examples} ; гарантии m_{tables} , m_{examples} ; порог τ .

Выход: таблицы с DDL и примеры запросов $\mathcal{R}(q)$.

Описание. Для кодирования используется multilingual-e5-large [10] — мультиязычная модель, обрабатывающая русскоязычные запросы и схему без потери качества представления.

Вектор запроса нормализуется до единичной нормы, после чего выполняется поиск по Inner Product, эквивалентный косинусному сходству. Результаты фильтруются порогом $\tau = 0.12$: документы ниже этого значения отбрасываются как семантически нерелевантные. Оставшиеся нормализуются относительно максимального score внутри каждого индекса:

$$\tilde{s}_i = \frac{s_i}{s_{\max}}.$$

Нормализация относительно максимума, а не диапазона $[\min, \max]$, сохраняет абсолютную уверенность поиска: если все документы индекса слабо похожи на запрос, их нормализованные оценки останутся низкими и не вытеснят документы из другого индекса с более высоким сходством.

Результаты обоих индексов объединяются взвешенным слиянием: итоговый score документа равен $w \cdot \tilde{s}$. После сортировки применяются минимальные гарантии покрытия: если в топ- k попало меньше m_{tables} таблиц или меньше m_{examples} примеров, недостающие добираются из хвоста соответствующего индекса независимо от их score. Гарантия на таблицы страхует от ситуации когда примеры полностью вытесняют структурный контекст.

4.5 Граф схемы и структурное обогащение

Семантический поиск решает задачу релевантности, но не связности: найденные таблицы не обязательно образуют корректный JOIN-путь. Между ними могут быть промежуточные таблицы, которые ретривал не нашёл, потому что в вопросе о них ничего нет. Например, запрос «топ пользователей по числу соревнований» семантически близок к `user` и `competition`, но для их соединения необходима `participation`, которая в вопросе не упоминается. Без структурного знания о схеме модель либо галлюцинирует соединение напрямую, либо теряет связь между таблицами.

Граф решает эту проблему: он знает о схеме то, чего не знает ретривал — какие таблицы являются связующими звеньями и как именно они соединяются. Граф строится из `graph.yaml` при старте сервиса. Каждая таблица — узел с показателем центральности `hub_score`: чем он ниже, тем охотнее граф использует таблицу как связующее звено при построении JOIN-пути. Таблицы с `hub_score ≥ 5` — листья, они исключаются из автоматического FK-расширения (`file_artifact`, `prize`, `code_kernel`). Рёбра хранят направление связи, имена столбцов, кардинальность и предпочтительный тип JOIN.

Алгоритм 2. Структурное обогащение контекста

Вход: $\mathcal{T}_{\text{RAG}}$; граф $G = (V, E)$; DDL.

Выход: $\mathcal{T}_{\text{enriched}}$; JOIN-подсказка h .

Описание. Алгоритм работает в два шага. На первом выполняется FK-расширение: для каждой найденной таблицы рассматриваются все соседи по рёбрам внешних ключей и добавляются те, чей `hub_score < 5`. Так в контекст попадают промежуточные таблицы даже если в вопросе о них ничего нет.

На втором шаге по исходному набору таблиц RAG запускается алгоритм Дейкстры для построения оптимального JOIN-пути. Вес ребра отражает «стоимость» включения соединения в запрос:

$$w(u, v) = 1.0 + \delta_{\text{card}}(u, v) + \max\left(0, \frac{\text{hub}(u) + \text{hub}(v)}{2} - 1\right) \cdot c_{\text{step}},$$

где δ_{card} : 0.0 для «один к одному», 0.05 для «многие к одному», 0.10 для «один ко многим»; $c_{\text{step}} = 0.12$. Штраф за кардинальность заставляет алгоритм избегать рёбер с риском размножения строк перед агрегацией и предпочитать более безопасные пути соединения. Обратные рёбра добавляются автоматически с коэффициентом 1.5, чтобы предпочтительным направлением обхода оставалось исходное направление внешнего ключа.

Результат — упорядоченная цепочка таблиц с готовыми JOIN-путями, которая передаётся планировщику как подсказка. Максимальная длина пути ограничена шестью переходами; если путь не найден, генерация продолжается с результатами RAG.

4.6 Двухэтапная генерация SQL и роли языковой модели

Генерация разбита на два последовательных обращения к модели — это реализация Chain-of-Thought. Первое строит структурный план соединений, второе исполняет его в SQL. Порядок секций контекста фиксирован: FK-подсказка, DDL таблиц, примеры (не более трёх). Промпты написаны на русском: смешение языков нарушало формат вывода и порядок приоритетов.

Алгоритм 3. Двухэтапная генерация SQL

Вход: вопрос q ; $\mathcal{T}_{\text{enriched}}$; примеры E ; подсказка h .

Выход: SQL-запрос $\hat{s}$.

Роль 1 — планировщик.

Ты - планировщик SQL для PostgreSQL.

Построй минимальный и корректный JOIN-план и верни ТОЛЬКО JSON.

Используй контекстные блоки:

- "## Подсказка FK-пути" - базовый маршрут JOIN.
- "## Доступные таблицы" - разрешенные таблицы и столбцы.
- "## Похожие примеры" - шаблоны запросов.

Правила:

1. Не используй таблицы и столбцы, которых нет в

"## Доступные таблицы".

2. Покрой все фильтры из вопроса: для каждого фильтра должна быть таблица в плане.
3. Следуй FK-пути по возможности; добавляй только нужные JOIN.
4. Для агрегатов соблюдай гранулярность: агрегируй на уровне сущности из вопроса и не допускай размножения строк перед AVG/SUM/COUNT/MIN/MAX.
5. План должен быть простым: без лишних таблиц и связей.

Формат: {"from_clause":"...", "joins":["..."],
"select_hint":"...", "where_hints":["..."]}

Выводи ТОЛЬКО JSON без маркдауна и пояснений.

При невалидном JSON план пропускается, генерация переходит к роли 2 без структурного плана.

Роль 2 — генератор SQL.

Ты - эксперт PostgreSQL.

Напиши корректный и короткий SQL по вопросу, используя только предоставленный контекст.

Правила:

1. Только таблицы и столбцы из "## Доступные таблицы".
2. FROM/JOIN из "## Подтверждённый план JOIN".
Если блока нет - из "## Подсказка FK-пути".
3. Не добавляй лишние JOIN: каждая таблица нужна для SELECT, WHERE, GROUP BY или ORDER BY.
4. Для агрегатов соблюдай гранулярность: агрегируй на уровне сущности из вопроса и не допускай размножения строк перед AVG/SUM/COUNT/MIN/MAX.
5. Без CTE, подзапросов и оконных функций, если не требует вопрос.
6. Короткие уникальные алиасы таблиц.

ВЫВОД: только SQL, без маркдауна.

Каждый clause - с новой строки.

Температура 0.0. Планировщик — 400 токенов, генератор — 600. При отрицательном вердикте судьи генерация повторяется с промптом коррекции, который передаёт описание ошибки и требует исправить только её.

Роль 3 — судья. Оценивает корректность SQL и возвращает {valid, score, error, comments}. В текущей версии роль реализована, но отключена.

Ты - эксперт по проверке SQL-запросов.

Критерии valid=true (ВСЕ должны выполняться):

1. SQL реализует намерение пользователя.
2. Используются правильные таблицы и столбцы.
3. SELECT возвращает именно то, что запрашивал пользователь (правильная проекция).

НЕ являются основанием для valid=false:

- точка с запятой в конце,
- порядок столбцов в условии ON,
- алиасы столбцов (AS avg_score и т.п.),
- незначительные стиливые отличия от эталона.

Шкала score:

0.95-1.00: семантически эквивалентный SQL.

0.85-0.94: корректный, небольшие неоптимальности.

0.70-0.84: частично корректно, результат зависит от данных.

0.40-0.69: существенные ошибки.

0.00-0.39: неверный SQL по смыслу или структуре.

Если valid=false, score < 0.85.

Верни JSON: valid (bool), score (0-1), error, comments.

Выводи ТОЛЬКО JSON, без пояснений и маркдауна.

Список литературы

- [1] Lewis P., Perez E., Piktus A., et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks // *Advances in Neural Information Processing Systems (NeurIPS)*. — 2020. — Vol. 33. — P. 9459–9474.
- [2] Wei J., Wang X., Schuurmans D., et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models // *Advances in Neural Information Processing Systems (NeurIPS)*. — 2022. — Vol. 35. — P. 24824–24837.
- [3] Kojima T., Gu S. S., Reid M., Matsuo Y., Iwasawa Y. Large Language Models are Zero-Shot Reasoners // *Advances in Neural Information Processing Systems (NeurIPS)*. — 2022. — Vol. 35. — P. 22199–22213.
- [4] Gao Y., Xiong Y., Gao X., et al. Retrieval-Augmented Generation for Large Language Models: A Survey // *arXiv preprint arXiv:2312.10997*. — 2023.
- [5] Zhong V., Xiong C., Socher R. Seq2SQL: Generating Structured Queries from Natural Language Using Reinforcement Learning // *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. — 2017. — P. 1073–1084.
- [6] Affolter K., Stockinger K., Bernstein A. A Comparative Survey of Recent Natural Language Interfaces for Databases // *The VLDB Journal*. — 2019. — Vol. 28. — P. 793–819.
- [7] Li J., Hui B., Qu G., et al. Can LLM Already Serve as a Database Interface? A BIG Bench for Large-Scale Database Grounded Text-to-SQLs // *Advances in Neural Information Processing Systems (NeurIPS)*. — 2023. — Vol. 36.
- [8] Zheng L., Chiang W., Sheng Y., et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena // *Advances in Neural Information Processing Systems (NeurIPS)*. — 2023. — Vol. 36.
- [9] FAISS: Facebook AI Similarity Search [Electronic resource]. — URL: <https://faiss.ai/>.
- [10] multilingual-e5-large: Multilingual Text Embeddings Model [Electronic resource]. — URL: <https://huggingface.co/intfloat/multilingual-e5-large>.
- [11] Ollama: Local Large Language Model Inference Engine [Electronic resource]. — URL: <https://ollama.com/>.
- [12] Sentence Transformers: Multilingual Sentence Embeddings Library [Electronic resource]. — URL: <https://www.sbert.net/>.
- [13] NetworkX: Network Analysis in Python [Electronic resource]. — URL: <https://networkx.org/>.